

РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ

Факультет физико-математических и естественных наук

Кафедра компьютерных и информационных наук

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 6

дисциплина: Архитектура компьютера

Студент: Симакова Алина

Группа: НКАбд-05-25

МОСКВА

2025 г.

Оглавление

1 Цель работы.....	3
2 Задание.....	4
3 Теоретическое введение.....	5
4 Выполнение лабораторной работы.....	6
4.1 Символьные и числительные данные в NASM.....	6
4.2 Выполнение арифметических операций в NASM.....	10
4.3 Ответы на контрольные вопросы.....	13
5 Задания для самостоятельной работы.....	14
6 Выводы.....	16
Список литературы.....	17

1 Цель работы

Целью данной лабораторной работы является освоение арифметических инструкций и языка ассемблера в NASM.

2 Задание

На основе методических указаний изучить символьные и численные данные в NASM, выполнить арифметические операции в NASM и выполнить задания для самостоятельной работы.

3 Теоретическое введение

Большинство инструкций на языке ассемблера требуют обработки операндов. Адрес операнда предоставляет место, где хранятся данные, подлежащие обработке. Это могут быть данные хранящиеся в регистре или в ячейке памяти.

Регистровая адресация – операнды хранятся в регистрах и в команде используются имена этих регистров, например: `mov ax,bx`.

Непосредственная адресация – значение операнда задается непосредственно в команде, например: `mov ax,2`.

Адресация памяти – операнд задает адрес в памяти. В команде указывается символическое обозначение ячейки памяти, над содержимым которой требуется выполнить операцию. Ввод информации с клавиатуры и вывод её на экран осуществляется в символьном виде. Кодирование этой информации производится согласно кодовой таблице символов ASCII. ASCII – сокращение от American Standard Code for Information Interchange (Американский стандартный код для обмена информацией). Согласно стандарту ASCII каждый символ кодируется одним байтом. Среди инструкций NASM нет такой, которая выводит числа (не в символьном виде). Поэтому, например, чтобы вывести число, надо предварительно преобразовать его цифры в ASCII-коды этих цифр и выводить на экран эти коды, а не само число. Если же выводить число на экран непосредственно, то экран воспримет его не как число, а как последовательность ASCII-символов – каждый байт числа будет воспринят как один ASCII-символ – и выведет на экран эти символы.

Аналогичная ситуация происходит и при вводе данных с клавиатуры. Введенные данные будут представлять собой символы, что сделает невозможным получение корректного результата при выполнении над ними арифметических операций. Для решения этой проблемы необходимо проводить преобразование ASCII символов в числа и обратно.

4 Выполнение лабораторной работы

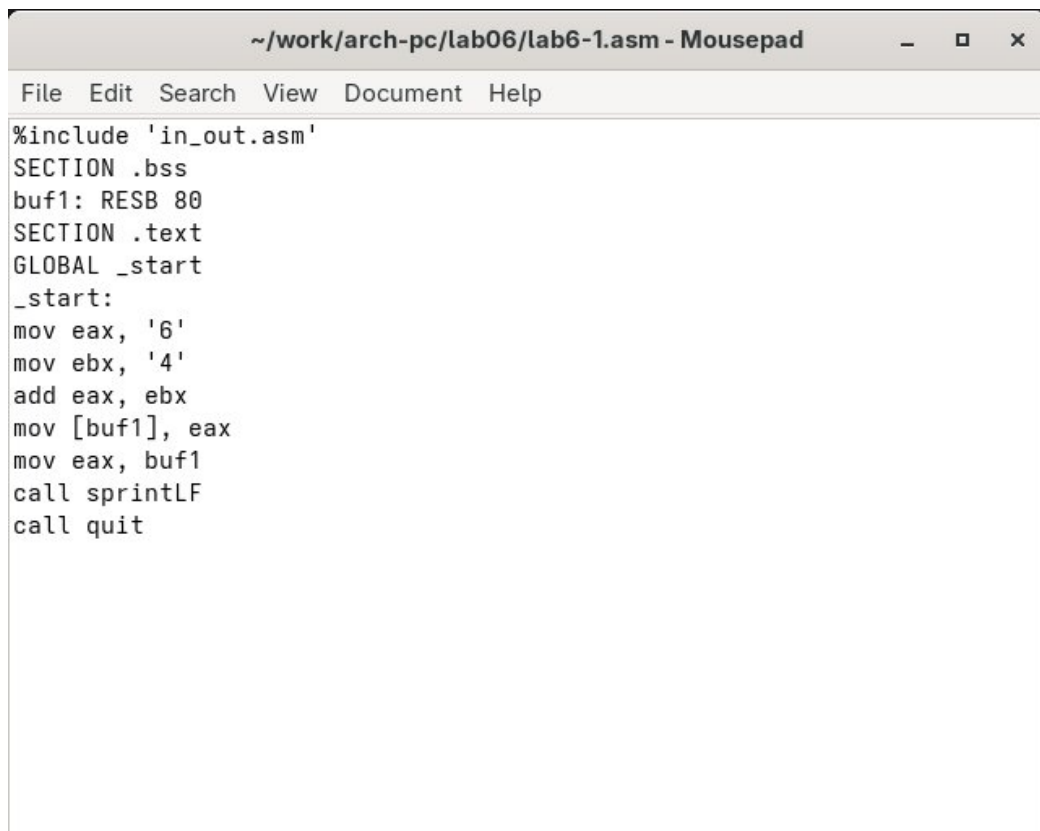
4.1 Символьные и численные данные в NASM

В домашней директории создаю каталог для программ лабораторной работы №6 и перехожу в него, создаю там файл (рис. 4.1).

```
simakovaalina@fedora:~$ mkdir ~/work/arch-pc/lab06
simakovaalina@fedora:~$ cd ~/work/arch-pc/lab06
simakovaalina@fedora:~/work/arch-pc/lab06$ touch lab6-1.asm
simakovaalina@fedora:~/work/arch-pc/lab06$
```

Рис. 4.1: Создание нового каталога

В созданном файле ввожу программу из листинга (рис. 4.2).



```
%include 'in_out.asm'
SECTION .bss
buf1: RESB 80
SECTION .text
GLOBAL _start
_start:
mov eax, '6'
mov ebx, '4'
add eax, ebx
mov [buf1], eax
mov eax, buf1
call sprintLF
call quit
```

Рис. 4.2: Сохранение новой программы

Создаю исполняемый файл и запускаю его. Вывод программы отличается от предполагаемого изначально, так как коды символов в сумме дают символ j по таблице ASCII (рис. 4.3).

```
simakovaalina@fedora:~/work/arch-pc/lab06$ mousepad lab6-1.asm
simakovaalina@fedora:~/work/arch-pc/lab06$ nasm -f elf lab6-1.asm
simakovaalina@fedora:~/work/arch-pc/lab06$ ld -m elf_i386 -o lab6-1.o
ld: no input files
simakovaalina@fedora:~/work/arch-pc/lab06$ ld -m elf_i386 -o lab6-1 lab6-1.o
simakovaalina@fedora:~/work/arch-pc/lab06$ ./lab6-1
j
simakovaalina@fedora:~/work/arch-pc/lab06$
```

Рис. 4.3: Запуск изначальной программы

Далее изменяю текст изначальной программы, убрав кавычки (рис. 4.4).

```
simakovaalina@fedora:~/work/arch-pc/lab06$ cat lab6-1.asm
#include 'in_out.asm'

SECTION .bss
buf1: RESB 80

SECTION .text
GLOBAL _start
_start:

mov eax, 6
mov ebx, 4
add eax, ebx
mov [buf1], eax
mov eax, buf1
call sprintf

call quitsimakovaalina@fedora:~/work/arch-pc/lab06$
```

Рис. 4.4: Измененная программа

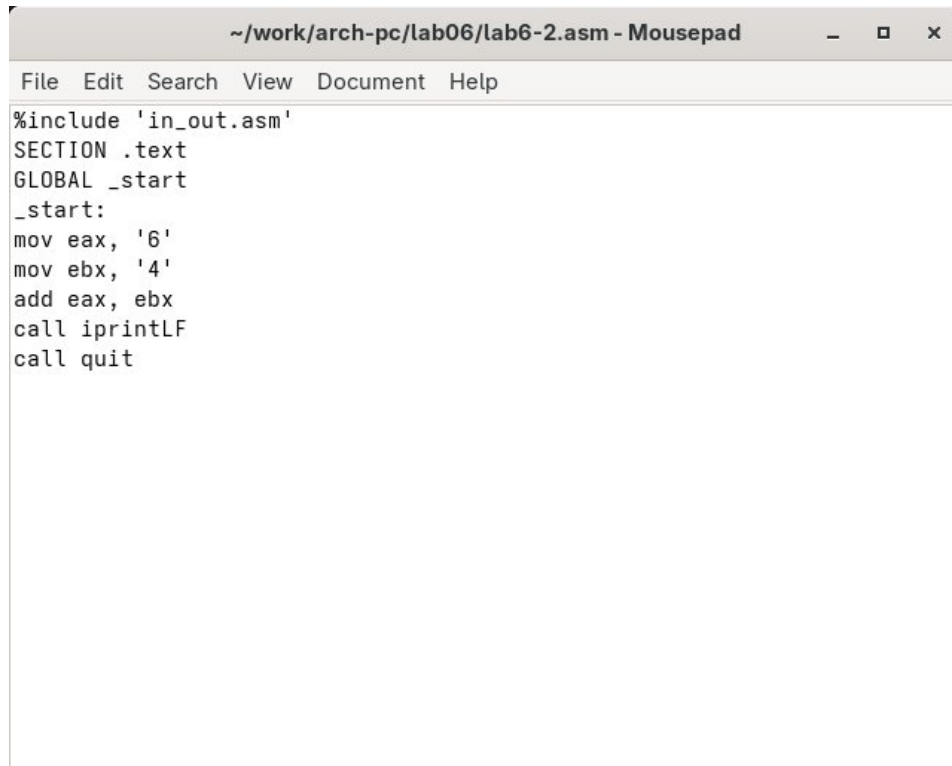
Теперь программа выдала пустую строку. Это связано с тем, что символ 10 означает переход на новую строку (рис. 4.5).

```
simakovaalina@fedora:~/work/arch-pc/lab06$ nasm -f elf lab6-1.asm
simakovaalina@fedora:~/work/arch-pc/lab06$ ld -m elf_i386 -o lab6-11 lab6-1.o
simakovaalina@fedora:~/work/arch-pc/lab06$ ./lab6-11

simakovaalina@fedora:~/work/arch-pc/lab06$
```

Рис. 4.5: Запуск измененной программы

Создаю файл для будущей программы и записываю в неё код из листинга (рис. 4.6)



```
~/work/arch-pc/lab06/lab6-2.asm - Mousepad
File Edit Search View Document Help
%include 'in_out.asm'
SECTION .text
GLOBAL _start
_start:
mov eax, '6'
mov ebx, '4'
add eax, ebx
call iprintLF
call quit
```

Рис. 4.6: Вторая программа

Создаю исполняемый файл и запускаю его. Отображается результат 106, программа, как и в первый раз, сложила коды символов, но вывела само число, а не его символ, благодаря замене функции вывода на iprintLF (рис. 4.7).



```
simakovaalina@fedora:~/work/arch-pc/lab06$ touch lab6-2.asm
simakovaalina@fedora:~/work/arch-pc/lab06$ mousepad lab6-2.asm
simakovaalina@fedora:~/work/arch-pc/lab06$ nasm -f elf lab6-2.asm
simakovaalina@fedora:~/work/arch-pc/lab06$ ld -m elf_i386 -o lab6-2 lab6-2.o
simakovaalina@fedora:~/work/arch-pc/lab06$ ./lab6-2
106
simakovaalina@fedora:~/work/arch-pc/lab06$
```

Рис. 4.7: Вывод второй программы

Убрав кавычки в программе, я снова её запускаю и получаю предполагаемый изначально результат (рис. 4.8).


```

simakovaalina@fedora:~/work/arch-pc/lab06$ mousepad lab6-2.asm
simakovaalina@fedora:~/work/arch-pc/lab06$ cat lab6-2.asm
#include 'in_out.asm'
SECTION .text
GLOBAL _start
_start:
mov eax, 6
mov ebx, 4
add eax, ebx
call iprintLF
call quitsimakovaalina@fedora:~/work/arch-pc/lab06$ nasm -f elf lab6-2.asm
simakovaalina@fedora:~/work/arch-pc/lab06$ ld -m elf_i386 -o lab6-21 lab6-2.o
simakovaalina@fedora:~/work/arch-pc/lab06$ ./lab6-21
10
simakovaalina@fedora:~/work/arch-pc/lab06$

```

Рис. 4.8: Вывод измененной второй программы

Заменив функцию вывода на `iprint`, я получаю тот же результат, но без переноса строки (рис. 4.9).

```

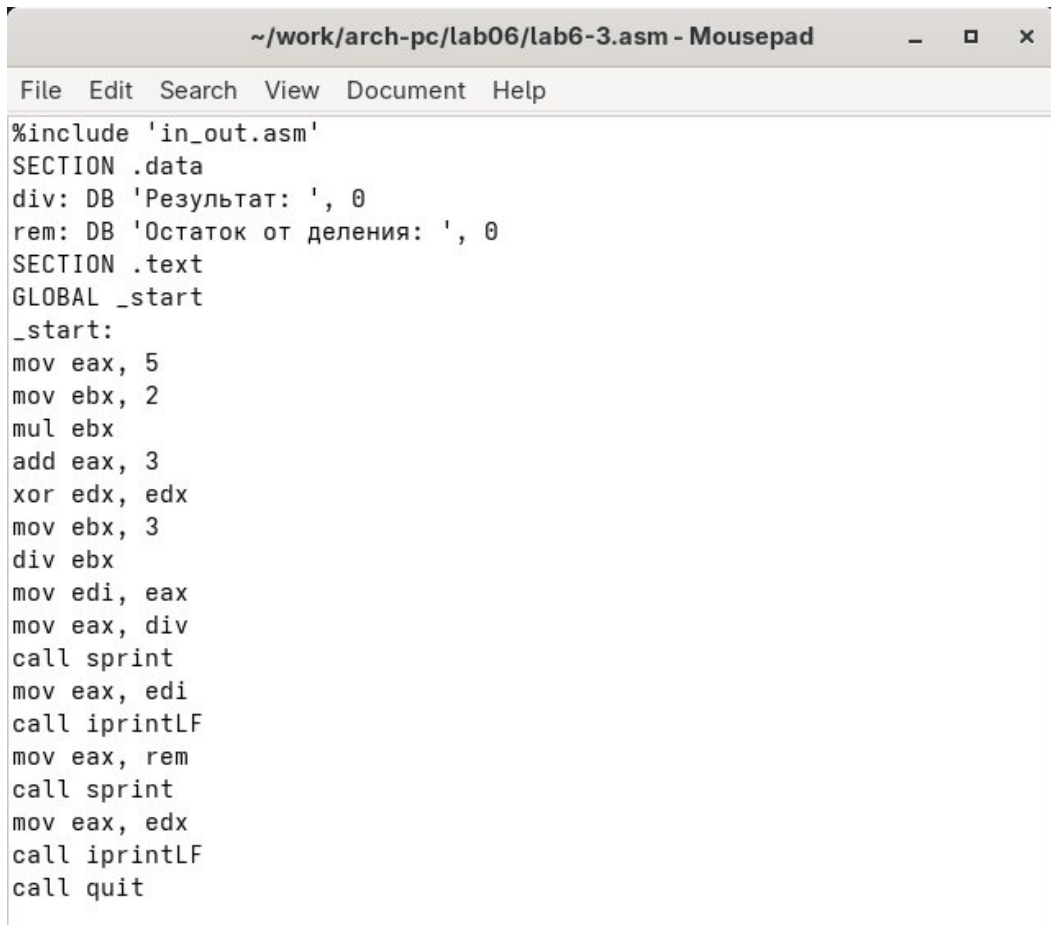
simakovaalina@fedora:~/work/arch-pc/lab06$ mousepad lab6-2.asm
simakovaalina@fedora:~/work/arch-pc/lab06$ cat lab6-2.asm
#include 'in_out.asm'
SECTION .text
GLOBAL _start
_start:
mov eax, 6
mov ebx, 4
add eax, ebx
call iprint
call quitsimakovaalina@fedora:~/work/arch-pc/lab06$ nasm -f elf lab6-2.asm
simakovaalina@fedora:~/work/arch-pc/lab06$ ld -m elf_i386 -o lab6-22 lab6-2.o
simakovaalina@fedora:~/work/arch-pc/lab06$ ./lab6-22
10simakovaalina@fedora:~/work/arch-pc/lab06$

```

Рис. 4.9: Замена функции вывода во второй программе

4.2 Выполнение арифметических операций в NASM

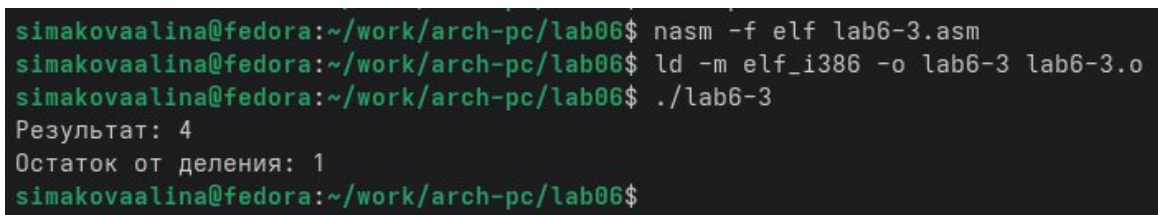
Создаю новый файл и копирую в него содержимое листинга (рис. 4.10).



```
~/work/arch-pc/lab06/lab6-3.asm - Mousepad
File Edit Search View Document Help
%include 'in_out.asm'
SECTION .data
div: DB 'Результат: ', 0
rem: DB 'Остаток от деления: ', 0
SECTION .text
GLOBAL _start
_start:
mov eax, 5
mov ebx, 2
mul ebx
add eax, 3
xor edx, edx
mov ebx, 3
div ebx
mov edi, eax
mov eax, div
call sprint
mov eax, edi
call iprintLF
mov eax, rem
call sprint
mov eax, edx
call iprintLF
call quit
```

Рис. 4.10: Третья программа

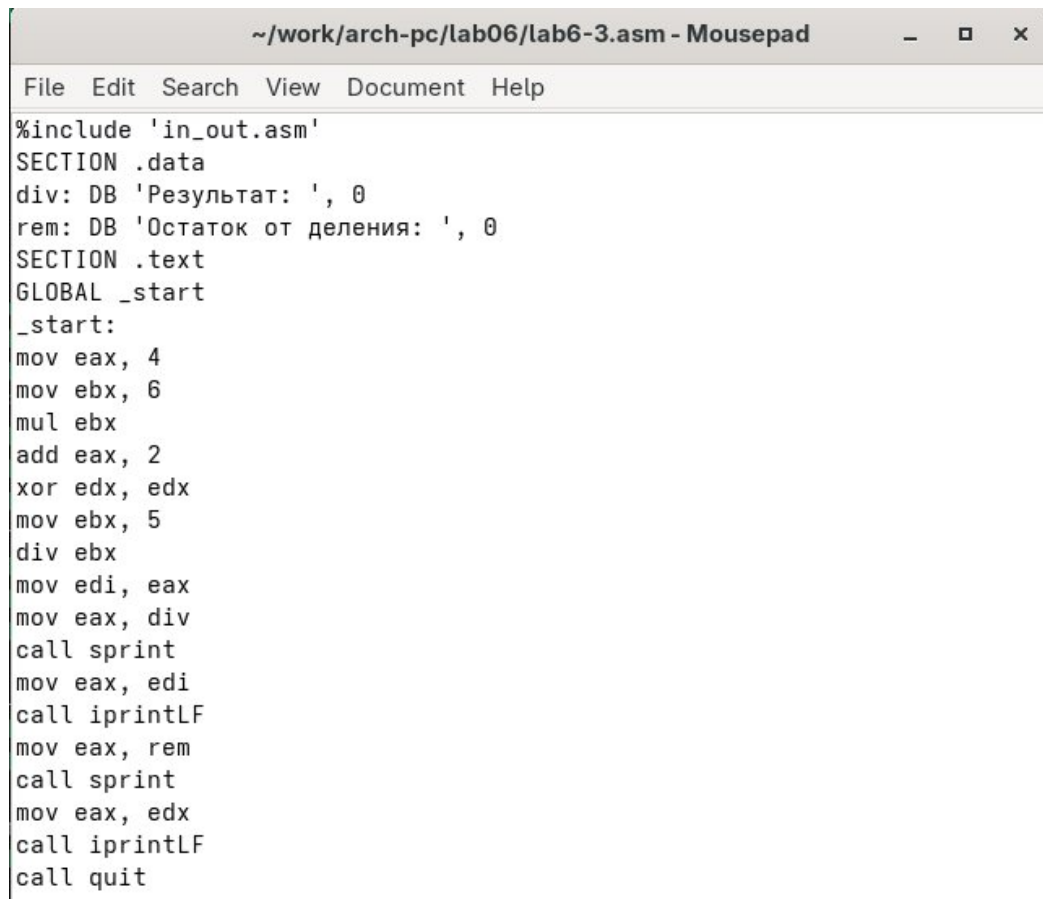
Затем Программа выполняет арифметические вычисления, на вывод идёт результирующее выражение и его остаток от деления (рис. 4.11).



```
simakovaalina@fedora:~/work/arch-pc/lab06$ nasm -f elf lab6-3.asm
simakovaalina@fedora:~/work/arch-pc/lab06$ ld -m elf_i386 -o lab6-3 lab6-3.o
simakovaalina@fedora:~/work/arch-pc/lab06$ ./lab6-3
Результат: 4
Остаток от деления: 1
simakovaalina@fedora:~/work/arch-pc/lab06$
```

Рис. 4.11: Запуск третьей программы

Заменяв переменные в программе для выражения $f(x) = (4*6+2)/5$ (рис. 4.12).



```
~/work/arch-pc/lab06/lab6-3.asm - Mousepad
File Edit Search View Document Help
%include 'in_out.asm'
SECTION .data
div: DB 'Результат: ', 0
rem: DB 'Остаток от деления: ', 0
SECTION .text
GLOBAL _start
_start:
mov eax, 4
mov ebx, 6
mul ebx
add eax, 2
xor edx, edx
mov ebx, 5
div ebx
mov edi, eax
mov eax, div
call sprint
mov eax, edi
call iprintLF
mov eax, rem
call sprint
mov eax, edx
call iprintLF
call quit
```

Рис. 4.12: Изменение третьей программы

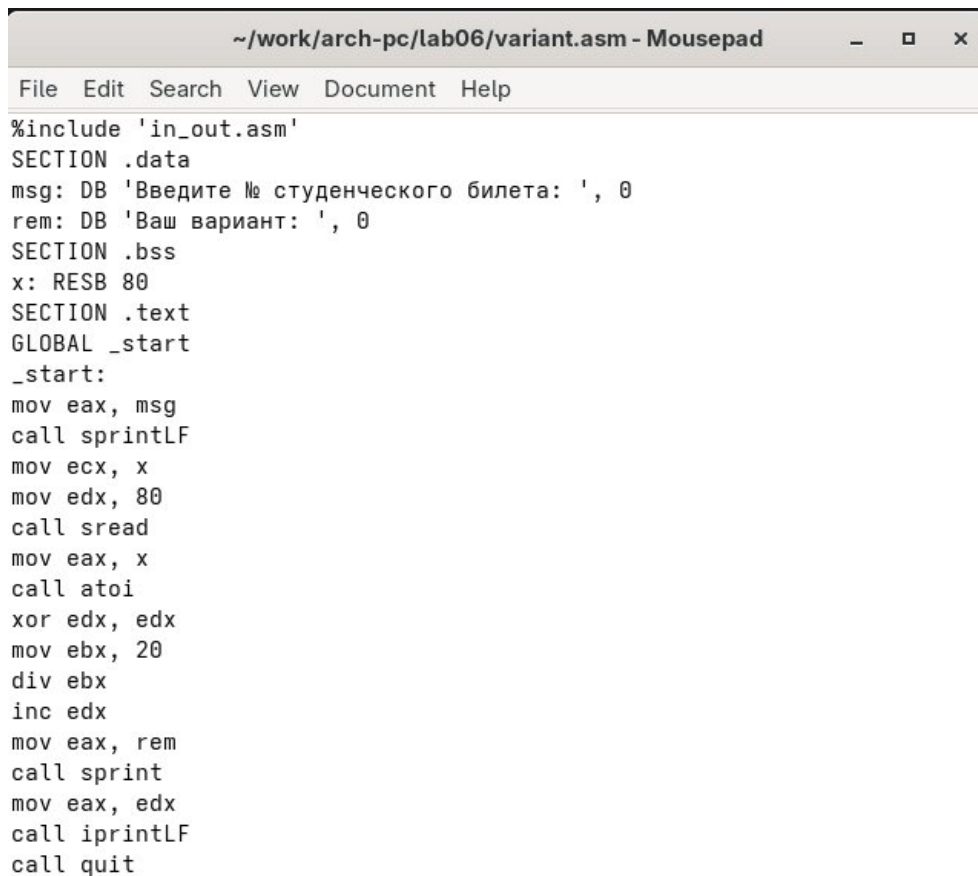
Запуск программы даёт корректный результат (рис. 4.13).



```
simakovaalina@fedora:~/work/arch-pc/lab06$ nasm -f elf lab6-3.asm
simakovaalina@fedora:~/work/arch-pc/lab06$ ld -m elf_i386 -o lab6-31 lab6-3.o
simakovaalina@fedora:~/work/arch-pc/lab06$ ./lab6-31
Результат: 5
Остаток от деления: 1
simakovaalina@fedora:~/work/arch-pc/lab06$
```

Рис. 4.13: Запуск изменённой третьей программы

Создаю новый файл и помещаю текст из листинга (рис. 4.14).



```
~/work/arch-pc/lab06/variant.asm - Mousepad
File Edit Search View Document Help
%include 'in_out.asm'
SECTION .data
msg: DB 'Введите № студенческого билета: ', 0
rem: DB 'Ваш вариант: ', 0
SECTION .bss
x: RESB 80
SECTION .text
GLOBAL _start
_start:
mov eax, msg
call sprintLF
mov ecx, x
mov edx, 80
call sread
mov eax, x
call atoi
xor edx, edx
mov ebx, 20
div ebx
inc edx
mov eax, rem
call sprint
mov eax, edx
call iprintLF
call quit
```

Рис. 4.14: Программа для подсчёта варианта

Запустив программу и указав свой номер студенческого билета, я получила свой вариант для дальнейшей работы (рис. 4.15).



```
simakovaalina@fedora:~/work/arch-pc/lab06$ touch variant.asm
simakovaalina@fedora:~/work/arch-pc/lab06$ mousepad variant.asm
simakovaalina@fedora:~/work/arch-pc/lab06$ nasm -f elf variant.asm
simakovaalina@fedora:~/work/arch-pc/lab06$ ld -m elf_i386 -o variant variant.o
simakovaalina@fedora:~/work/arch-pc/lab06$ ./variant
Введите № студенческого билета:
1032253560
Ваш вариант: 1
simakovaalina@fedora:~/work/arch-pc/lab06$
```

Рис. 4.15: Запуск программы для подсчёта варианта

4.3 Ответы на контрольные вопросы

1. За вывод сообщения “Ваш вариант” отвечают строки кода:

```
mov eax,rem
```

```
call sprint
```

2. Инструкция `mov ecx, x` используется, чтобы положить адрес вводимой строки `x` в регистр `ecx` `mov edx, 80` - запись в регистр `edx` длины вводимой строки `call sread` - вызов подпрограммы из внешнего файла, обеспечивающей ввод сообщения с клавиатуры.

3. `call atoi` используется для вызова подпрограммы из внешнего файла, которая преобразует `ascii`-код символа в целое число и записывает результат в регистр `eax`.

4. За вычисления варианта отвечают строки:

```
xor edx,edx ; обнуление edx для корректной работы div
```

```
mov ebx, 20 ; ebx = 20
```

```
div ebx ; eax = eax/20, edx - остаток от деления
```

```
inc edx ; edx = edx + 1
```

5. При выполнении инструкции `div ebx` остаток от деления записывается в регистр `edx`.

6. Инструкция `inc edx` увеличивает значение регистра `edx` на 1.

7. За вывод на экран результатов вычислений отвечают строки:

```
mov eax,edx
```

```
call iprintLF
```

5 Задания для самостоятельной работы

В соответствии с выбранным вариантом, я реализую программу для подсчета функции $f(x) = (10 + 2x)/3$, проверка на нескольких переменных показывает корректное выполнение программы (рис. 5.1).

```
simakovaalina@fedora:~/work/arch-pc/lab06$ touch lab6-var1.asm
simakovaalina@fedora:~/work/arch-pc/lab06$ mouspad lab6-var1.asm
bash: mouspad: command not found...
simakovaalina@fedora:~/work/arch-pc/lab06$ mousepad lab6-var1.asm
simakovaalina@fedora:~/work/arch-pc/lab06$ cat lab6-var1.asm
%include 'in_out.asm'
SECTION .data
msg: DB 'Введите значение переменной x: ',0
rem: DB 'Результат: ',0
SECTION .bss
x: RESB 80
SECTION .text
GLOBAL _start
_start:
mov eax, msg
call sprint
mov ecx, x
mov edx, 80
call sread
mov eax, x
call atoi
mov ebx, 2
mul ebx
add eax, 10
mov ebx, 3
div ebx
mov edi, eax
mov eax, rem
call sprint
mov eax, edi
call iprintLF
call quitsimakovaalina@fedora:~/work/arch-pc/lab06$ nasm -f elf lab6-var1.asm
simakovaalina@fedora:~/work/arch-pc/lab06$ ld -m elf_i386 -o lab6-var1 lab6-var1.o
simakovaalina@fedora:~/work/arch-pc/lab06$ ./lab6-var1
Введите значение переменной x: 1
Результат: 4
simakovaalina@fedora:~/work/arch-pc/lab06$ ./lab6-var1
Введите значение переменной x: 10
Результат: 10
simakovaalina@fedora:~/work/arch-pc/lab06$
```

Рис. 5.1: Запуск и проверка программы

Прикладываю код программы:

```
%include 'in_out.asm'
SECTION .data
msg: DB 'Введите значение переменной x: ',0
rem: DB 'Результат: ',0
SECTION .bss
x: RESB 80
SECTION .text
GLOBAL _start
_start:
mov eax, msg
call sprint
mov ecx, x
mov edx, 80
call sread
mov eax, x
call atoi
mov ebx, 2
mul ebx
add eax, 10
mov ebx, 3
div ebx
mov edi, eax
mov eax, rem
call sprint
mov eax, edi
call iprintLF
call quit
```

6 Выводы

При выполнении данной лабораторной работы я освоила арифметические инструкции языка ассемблера NASM .

Список литературы

1. https://esystem.rudn.ru/pluginfile.php/2088992/mod_resource/content/0/%D0%9F%D1%80%D0%B8%D0%BC%D0%B5%D1%80%20%D0%BE%D1%84%D0%BE%D1%80%D0%BC%D0%BB%D0%B5%D0%BD%D0%B8%D1%8F%20%D0%BE%D1%82%D1%87%D0%B5%D1%82%D0%B0%20%D0%BF%D0%BE%20%D0%BB%D0%B0%D0%B1%D0%BE%D1%80%D0%B0%D1%82%D0%BE%D1%80%D0%BD%D0%BE%D0%B9%20%D1%80%D0%B0%D0%B1%D0%BE%D1%82%D0%B5.pdf
2. <https://esystem.rudn.ru/mod/page/view.php?id=1030492>
3. https://esystem.rudn.ru/pluginfile.php/2089086/mod_resource/content/0/%D0%9B%D0%B0%D0%B1%D0%BE%D1%80%D0%B0%D1%82%D0%BE%D1%80%D0%BD%D0%B0%D1%8F%20%D1%80%D0%B0%D0%B1%D0%BE%D1%82%D0%B0%20%E2%84%966.%20%D0%90%D1%80%D0%B8%D1%84%D0%BC%D0%B5%D1%82%D0%B8%D1%87%D0%B5%D1%81%D0%BA%D0%B8%D0%B5%20%D0%BE%D0%BF%D0%B5%D1%80%D0%B0%D1%86%D0%B8%D0%B8%20%D0%B2%20NASM..pdf
4. <https://esystem.rudn.ru/mod/resource/view.php?id=1030496>